

Flood Node Firmware Complete Listings

The two Arduino sketches that run the system: one for every field node, one for the gateway.

These are the same listings printed on pages 11–15 of *Flood Node Circuit Diagrams & Pin Connections*, gathered here on their own so the code can be printed, reviewed or circulated without the drawings.

SKETCH	RUNS ON	HOW MANY	WHAT IT DOES
node.ino	ESP32 in the field node	One per station	Wakes every 5 minutes, measures stage and rainfall, transmits 13 bytes, sleeps
gateway.ino	ESP32 at the gateway	One per cluster	Listens continuously, validates, de-duplicates, writes to SD, publishes over MQTT

Change these three lines per station, nothing else

CONSTANT	WHERE IT COMES FROM
NODE_ID	Your own numbering, 1–254. Must be unique across the cluster.
DATUM_MM	Staff gauge reading + reported distance, measured on installation day
MM_PER_TIP	The 200 ml pour test: 0.2×32 / (tips actually counted)

FOLDER NAMES MATTER IN THE ARDUINO IDE

A sketch must sit in a folder of the same name: node/node.ino and gateway/gateway.ino. Open either file and the IDE will offer to create the folder for you.

THE RADIO SETTINGS MUST MATCH EXACTLY

Frequency, spreading factor, bandwidth, coding rate and sync word are set in both sketches. If any one of them differs, the gateway hears nothing at all — no error, no warning, just silence.

1 • Sender firmware — node.ino

Flash onto every field node. One file, continuing across the next pages.

NODE/NODE.INO — PART 1 OF 3

```
// =====
// FLOOD NODE — SENDER (transmitter)
// ESP32-WROOM-32 + J5N-SR04T v2.0 + tipping bucket + DS18B20 + SX1276
// Arduino IDE. One file. Every cycle ends in deep sleep.
// =====

#include <SPI.h>
#include <LoRa.h>
#include <OneWire.h>
#include <DallasTemperature.h>
#include "driver/rtc_io.h"

// ----- per-node configuration: change these three per station ----
#define NODE_ID      1          // unique per node, 1..254
#define DATUM_MM      6250      // sensor face above datum, measured on site
#define MM_PER_TIP    0.2f      // from the rain gauge calibration (page 10)

#define SLEEP_SECONDS 300       // 5 minutes between transmissions

// ----- pin map (matches the tables on page 5) -----
#define PIN_TRIG      17        // TX2 on the silkscreen
#define PIN_ECHO      16        // RX2 on the silkscreen
#define PIN_RAIN      GPIO_NUM_33 // RTC pin - can wake the chip
#define PIN_TEMP       4
#define PIN_VBAT      35        // input-only, ADC1
#define LORA_SS        5
#define LORA_RST       14
#define LORA_DIO0      26

// ----- radio: must match the gateway exactly -----
#define LORA_FREQ      865100000L // IN865 licence-free band
#define LORA_SF        9
#define LORA_BW        125E3
#define LORA_SYNC      0x34

// ----- these survive deep sleep -----
RTC_DATA_ATTR uint16_t rainTips = 0;
RTC_DATA_ATTR uint16_t seq      = 0;

OneWire oneWire(PIN_TEMP);
DallasTemperature tempSensor(&oneWire);

// ----- CRC16-CCITT. The gateway runs the identical function -----
uint16_t crc16(const uint8_t *d, size_t n) {
    uint16_t crc = 0xFFFF;
    for (size_t i = 0; i < n; i++) {
        crc ^= (uint16_t)d[i] << 8;
        for (uint8_t b = 0; b < 8; b++)
            crc = (crc & 0x8000) ? (crc << 1) ^ 0x1021 : (crc << 1);
    }
    return crc;
}
```

```

// ----- one ultrasonic ping: returns echo width in microseconds ---
uint32_t pingOnce() {
    digitalWrite(PIN_TRIG, LOW); delayMicroseconds(4);
    digitalWrite(PIN_TRIG, HIGH); delayMicroseconds(10);
    digitalWrite(PIN_TRIG, LOW);
    return pulseIn(PIN_ECHO, HIGH, 40000UL); // 40 ms ceiling = about 6.8 m
}

int cmpU32(const void *a, const void *b) {
    uint32_t x = *(const uint32_t*)a, y = *(const uint32_t*)b;
    return (x > y) - (x < y);
}

// ----- median of 9 pings, corrected for air temperature -----
// A single ping lands on whatever the surface was doing at that instant.
// The median throws out both the wave crest and the spray dropout;
// a mean would not.
int16_t measureDistanceMM(float tempC) {
    uint32_t s[9];
    uint8_t n = 0;
    for (uint8_t i = 0; i < 9; i++) {
        uint32_t t = pingOnce();
        if (t > 0) s[n++] = t;
        delay(60); // let the echoes die away
    }
    if (n < 5) return -1; // too few valid returns
    qsort(s, n, sizeof(uint32_t), cmpU32);
    uint32_t median = s[n / 2];

    float c = 331.4f + 0.606f * tempC; // speed of sound, m/s
    float mm = (median * c) / 2000.0f; // us x m/s / 2 -> mm
    if (mm < 250.0f || mm > 4500.0f) return -1; // blind zone / out of range
    return (int16_t)(mm + 0.5f);
}

// ----- battery, through the 100k/100k divider on GPIO35 -----
uint16_t readBatteryMV() {
    analogSetPinAttenuation(PIN_VBAT, ADC_11db);
    uint32_t acc = 0;
    for (uint8_t i = 0; i < 16; i++) { acc += analogReadMilliVolts(PIN_VBAT); delay(2); }
    return (uint16_t)((acc / 16) * 2); // the divider halved it
}

// ----- build and send the 13-byte packet (format on page 2) -----
void sendPacket(int16_t levelMM, int8_t tempC, uint16_t battMV) {
    uint8_t p[13];
    p[0] = 1; // protocol version
    p[1] = NODE_ID;
    p[2] = seq >> 8; p[3] = seq & 0xFF;
    p[4] = levelMM >> 8; p[5] = levelMM & 0xFF;
    p[6] = rainTips >> 8; p[7] = rainTips & 0xFF;
    p[8] = battMV >> 8; p[9] = battMV & 0xFF;
    p[10] = (uint8_t)tempC;
    uint16_t c = crc16(p, 11);
    p[11] = c >> 8; p[12] = c & 0xFF;

    LoRa.beginPacket();
    LoRa.write(p, 13);
    LoRa.endPacket(); // blocks until sent
    seq++;
}

// ----- arm both wake sources and sleep -----
void goToSleep() {
    LoRa.sleep();
    rtc_gpio_pullup_en(PIN_RAIN); // hold the line high in sleep
    esp_sleep_enable_ext0_wakeup(PIN_RAIN, 0); // wake when a tip pulls it low
    esp_sleep_enable_timer_wakeup((uint64_t)SLEEP_SECONDS * 1000000ULL);
    esp_deep_sleep_start();
}

```

```

void setup() {
  pinMode(PIN_TRIG, OUTPUT);
  pinMode(PIN_ECHO, INPUT);
  pinMode(33, INPUT_PULLUP);           // no external pull-up fitted

  esp_sleep_wakeup_cause_t why = esp_sleep_get_wakeup_cause();

  // --- a rain tip: count it and go straight back to sleep -----
  if (why == ESP_SLEEP_WAKEUP_EXT0) {
    rainTips++;
    delay(150);                        // let the contact settle
    while (digitalRead(33) == LOW) delay(10); // wait for the bucket to finish
    delay(100);
    goToSleep();
  }

  // --- timer wake, or first boot: measure and transmit -----
  tempSensor.begin();
  tempSensor.requestTemperatures();
  float tC = tempSensor.getTempCByIndex(0);
  if (tC < -40 || tC > 80) tC = 25.0;   // probe missing -> assume 25 C

  int16_t dist = measureDistanceMM(tC);
  int16_t level = (dist < 0) ? -1 : (int16_t)(DATUM_MM - dist);

  SPI.begin(18, 19, 23, LORA_SS);
  LoRa.setPins(LORA_SS, LORA_RST, LORA_DI00);
  if (LoRa.begin(LORA_FREQ)) {
    LoRa.setSpreadingFactor(LORA_SF);
    LoRa.setSignalBandwidth(LORA_BW);
    LoRa.setCodingRate4(5);
    LoRa.setSyncWord(LORA_SYNC);
    LoRa.setTxPower(14);
    sendPacket(level, (int8_t)tC, readBatteryMV());
  }

  rainTips = 0;                        // counter reported, start the next interval
  goToSleep();
}

void loop() { }                       // never reached - every cycle ends in sleep

```

2 · Receiver firmware — gateway.ino

Flash onto the gateway ESP32. Never sleeps; parked in continuous receive.

GATEWAY/GATEWAY.INO — PART 1 OF 2

```
// =====
// FLOOD GATEWAY — RECEIVER
// ESP32-WROOM-32 + SX1276 + micro-SD. Never sleeps.
// Listens continuously, validates, de-duplicates, publishes, logs.
// =====

#include <WiFi.h>
#include <PubSubClient.h>
#include <SPI.h>
#include <LoRa.h>
#include <SD.h>

const char* WIFI_SSID = "site-wifi";
const char* WIFI_PASS = "*****";
const char* MQTT_HOST = "twin.example.in";
const int MQTT_PORT = 1883;
const char* GW_ID = "GW-KAMRUP-01";

#define LORA_SS 5
#define LORA_RST 14
#define LORA_DIO0 26
#define SD_CS 15 // shares SCK/MOSI/MISO with the radio

WiFiClient net;
PubSubClient mqtt(net);

uint16_t lastSeq[255]; // per-node duplicate filter
bool sdReady = false;
unsigned long lastBeat = 0;

// ----- identical to the node's function -----
uint16_t crc16(const uint8_t *d, size_t n) {
    uint16_t crc = 0xFFFF;
    for (size_t i = 0; i < n; i++) {
        crc ^= (uint16_t)d[i] << 8;
        for (uint8_t b = 0; b < 8; b++)
            crc = (crc & 0x8000) ? (crc << 1) ^ 0x1021 : (crc << 1);
    }
    return crc;
}

void connectAll() {
    if (WiFi.status() != WL_CONNECTED) {
        WiFi.begin(WIFI_SSID, WIFI_PASS);
        for (int i = 0; i < 40 && WiFi.status() != WL_CONNECTED; i++) delay(500);
    }
    if (!mqtt.connected()) mqtt.connect(GW_ID);
}
```

```

// ----- every payload also goes to the card, link up or down -----
void logToCard(const char *line) {
  if (!sdReady) return;
  File f = SD.open("/log.csv", FILE_APPEND);
  if (!f) return;
  f.println(line);
  f.close();
}

void setup() {
  Serial.begin(115200);
  memset(lastSeq, 0xFF, sizeof(lastSeq));

  // One SPI bus, two devices. Start the radio first, then the card.
  SPI.begin(18, 19, 23, LORA_SS);
  LoRa.setPins(LORA_SS, LORA_RST, LORA_DI00);
  while (!LoRa.begin(865100000L)) { Serial.println("LoRa init failed"); delay(2000); }
  LoRa.setSpreadingFactor(9);
  LoRa.setSignalBandwidth(125E3);
  LoRa.setCodingRate4(5);
  LoRa.setSyncWord(0x34);
  LoRa.receive(); // park in continuous receive

  sdReady = SD.begin(SD_CS, SPI);
  Serial.println(sdReady ? "SD ready" : "SD missing - buffering disabled");

  mqtt.setServer(MQTT_HOST, MQTT_PORT);
  connectAll();
}

void loop() {
  mqtt.loop();
  if (!mqtt.connected()) connectAll();

  // --- heartbeat, so a silent gateway is distinguishable from no rain -
  if (millis() - lastBeat > 60000UL) {
    lastBeat = millis();
    char t[48];
    snprintf(t, sizeof(t), "flood/%s/status", GW_ID);
    mqtt.publish(t, "{\\"alive\\":true}");
  }

  int sz = LoRa.parsePacket();
  if (sz != 13) return; // wrong length: not one of ours

  uint8_t p[13];
  for (int i = 0; i < 13; i++) p[i] = LoRa.read();

  uint16_t got = ((uint16_t)p[11] << 8) | p[12];
  if (got != crc16(p, 11)) return; // corrupt in the air, drop silently

  uint8_t node = p[1];
  uint16_t seq = ((uint16_t)p[2] << 8) | p[3];
  if (seq == lastSeq[node]) return; // duplicate
  lastSeq[node] = seq;

  int16_t level = (int16_t)(((uint16_t)p[4] << 8) | p[5]);
  uint16_t tips = ((uint16_t)p[6] << 8) | p[7];
  uint16_t batt = ((uint16_t)p[8] << 8) | p[9];
  int8_t tempC = (int8_t)p[10];

  char topic[48], payload[220];
  snprintf(topic, sizeof(topic), "flood/%s/node/%u", GW_ID, node);
  snprintf(payload, sizeof(payload),
    "{\\"node\\":%u,\\"seq\\":%u,\\"level_mm\\":%d,\\"rain_tips\\":%u,\\"rain_mm\\":%.1f,\\"batt_mv\\":%u,\\"temp_c\\":%d,\\"rssi\\":%d,\\"snr\\":%.1f}",
    node, seq, level, tips * 0.2f, batt, tempC,
    LoRa.packetRssi(), LoRa.packetSnr());

  mqtt.publish(topic, payload);
  logToCard(payload);
  Serial.println(payload);
}

```